06 NAREDBE LDM i STM

LDM → load multiple ⇒ puni u niz registera podatke pročitane iz niza uzastopnih riječi u memoriji

STM → store multiple ⇒ sprema podatke iz niza reg. uzastop. riječi u memo

- prenose se isključivo RIJEČI; 32-bitni podaci
- rad s blokovima podataka
- rad sa stogovima

LDM / STM <cond><mode_4> baza registar <!>, {lista registara}

LDMIA R2!, {R3, R8-R6, R1, R10-R12}

SA
- kon. adresa upisuje u baz registar

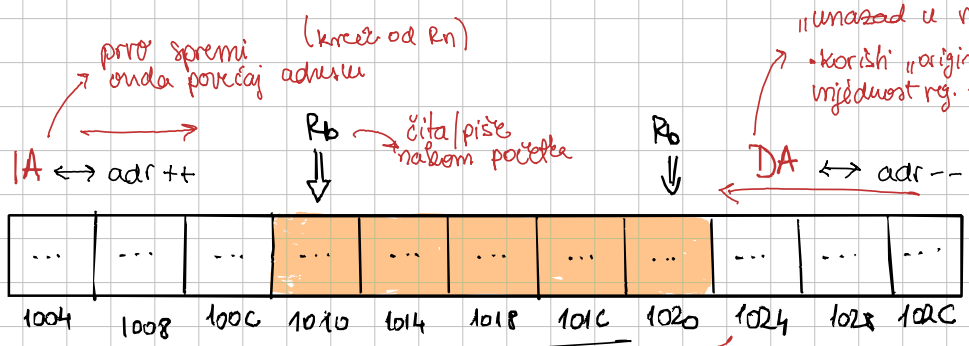
BEZ
- konačna adresa NE upisuje u baz reg. (bilo koji osim R15)

// u praksi za stog → R13

LDM i STM rad s blokovima

IA Increment After
IB Increment Before
} Rb pokazuje početak bloka memorije

DA Decrement After
DB Decrement Before
} Rb pokazuje kraj bloka memorije



I - povećaj adresu (kreći od Rn+4)

B - prije nego što spremiš

Cheat - sheet

MOD	Značenje	Raspored adr.	Auto-increment (!)	Smjer		Objašnjenje
IA	Increment After	! Donji → Gornji	POVEĆAVA NAKON	↑	(Rn)	poslije svakog pristupa povećava za 4 bajta
IB	Increment Before	Donji → Gornji	POVEĆAVA PRIJE	↑	(Rn+4)	prvo poveća
DA	Decrement After	Gornji → Donji	SMANJUJE NAKON	↓	(Rn)	smanjuje nakon pristupa
DB	Decrement Before	! Gornji → Donji	SMANJUJE PRIJE	↓	(Rn-4)	prvo smanji

Auto increment

→ STMIA Rn!, {...} ; nakon svakog storea, Rn se ažurira

→ LDMB Rn!, {...} ; nakon loada, Rn se smanjuje za n*4

Bez ! ⇒ Rn ostaje isti (koristimo ako ne želimo da se pointer pomakne)

Koji kod koristiti :

	tipično koristi:	Npr
Push na stack	STMDB sp!, {...}	spuštaš stack, spremaš
Pop sa stacka	LDMIA sp!, {...}	Dižeš stack, učitavaš
Kopiranje iz RAM-a	LDMIA Rn!, {...}	Čitaš niz podataka iz memorije i pomičeš Rn
Kopiranje u RAM	STMIA Rn!, {...}	Spremaš više registara u memoriju

Redoslijed registara iz liste u odnosu na niz podataka u memoriji je uvijek jednak

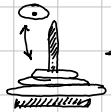
NIŽI INDEX ⇒ NIŽA ADRESA!

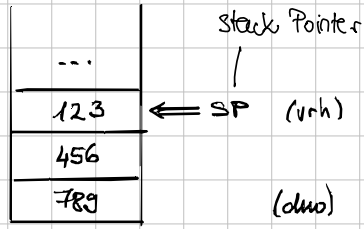
- ▷ bez obzira na redoslijed kojim su registri napisani u listi
- ▷ bez obzira koristi li se LDM ili STM
- ▷ bez obzira na način adresiranja IA, IB, DA, DB

STOG – područje memorije u koje se podatci pišu i čitaju

po LIFO redoslijedu

Last In First Out

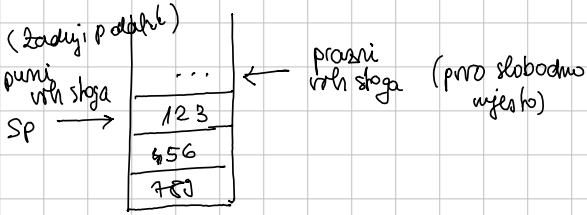




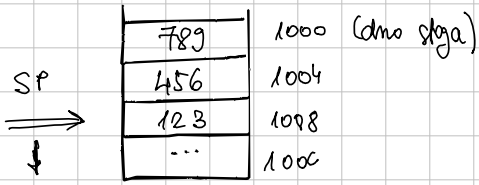
- Stack Pointer pokazuje na vrh stoga

Push → stavlja na vrh stoga

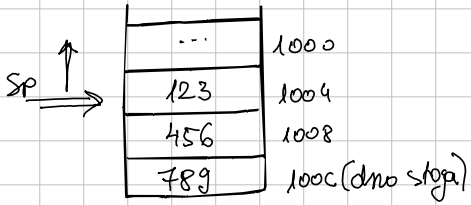
Pop → uzima s vrha stoga



Rastući stog



Padajući stog



u praksi češći

- uobičajeno smješten na najvišim mem. adresama

▶ LDM i STM kao PUSH i POP

- Popunjenost s vrha stoga
  - EMPTY
  - FULL
- Smjer porasta stoga
  - ASCENDING
  - DESCENDING

ED
FD
EA
FA

Puni vrh stoga

PUSH = STM FD

POP = LDM FD

Prazni vrh stoga

PUSH = STM ED

POP = LDM ED

Stog u programima

- SP se prije korištenja obavezno mora inicijalizirati
- paziti da uvijek skinemo sve podatke koje je stavio na stog

→ ne smiju se skidati nepostojeći podatci sa stoga

- FD stog => poč. vrijednost za ED je 0x10000 ----> ispravni bajt
  => poč. vrijednost za ED je 0XFFFC ----> nije ispravni bajt

07 FUNKCIJE

Pozivatelj funkcije

BL F1 ; poziv

branch and link

*dodatno sprema sadržaj PC-a (R15) u LR (R14)

Funkcija: F1

```
....  ; spremi kontekst
....  ; stvori lokalne varijable
....  ; dohvati argumente

....  ; koristne naredbe

....  ; pripremi rezultat
....  ; ukloni lokalne varijable
....  ; obnovi kontekst
MOV PC, LR ; povratak
```

! funkcija ne smije mijenjati ništa osim rezultata

! pohranjena se adresa vraća iz LR u PC !

Prijenos registrima

```
MAIN    MOV  R0, #35        ; pripremi argumente u R0 i R1
        MOV  R1, #12

        BL   RAZ_KVAD       ; >>> ! POZIV FUNKCIJE !

        STR  R0, REZ
        SWI  0x123456

REZ     DW   0              ; mjesto za rezultat

RAZ_KVAD
        MUL  R2, R0, R0     ; izravna upotreba parametara iz R0 i R1
        MUL  R0, R1, R1
        SUB  R0, R2, R0     ; izravno spremanje rez u R0

        MOV  PC, LR         ; povratak u pozivatelja
```

! VRLO LOŠE (mijenja R2; R0 je okej mijenjati jer i tako vraća

KONTEKST

→ svi REGISTRI koje funkcija mijenja, ne računajući REG. za koje pozivatelj zna da će biti promijenjeni, a to su:

▷ registri preko kojih se vraća REZULTAT → u njima pozivatelj ne smije imati korisne vrijednosti jer zna da će ih fja promijeniti

▷ registar R15 koji se inačko stalno mijenja

▷ registar CPSR jer mnoge naredbe na njega utječu pa se očekuje da može biti promijenjen

R15 se 100% mijenja
CPSR se vrlo vjr mijenja

→ ako fja mijenja registre kojima prima parametre → oni su isto dio konteksta

- kontekst SPREMAMO NA STOG

⇒ kontekst funkcije obavezno moramo uvijek spremati!

```
MAIN  MOV SP, #0x10000
      MOV R0, #35
      MOV R1, #12
      BL  RAZ_KVAD
      STR R0, REZ
      SWI 0x123456
REZ   DW  0
```

R2 ima vrijednost 0

R2 vrijednost i dalje 0 • mora ostati ∅ nakon fje

```
RAZ_KVAD
      STMFD SP!, {R2}   ; pohrani
      MUL   R2, R0, R0
      MUL   R0, R1, R1
      SUB   R0, R2, R0
      LMFD  SP!, {R2}   ; obnovi
      MOV   PC, LR
```

čuva R2

SP spušta R2 na stog → spašava kontekst

vraća originalan R2 sa stoga

Prijenos fiksnim lokacijama

- sve analogno kao za registre, jedino se umjesto reg. koristi fiksne lokacije u memoriji

memorijske lokacije kojima se pristupa pomoću labela

→ praktički neograničen broj parametara i rezultata

⟹ SPORIJI I DULJI NAČIN

- ONEMOGUĆUJE rekurzivne fje → rek. poziv uništio bi parametre iz prethodnog poziva

Prijenos stogom

: sporiji i duljі način jer traži dod naredbe LDM/SPM

→ vraćanje rez. stogom nije praktično ⟹ vraćanje registrom

→ OMOGUĆUJE rekurzivne pozive

- neograničen broj parametara i rezultata

PRIJENOS LOKACIJAMA IZA NAREDBE BL

→ neposredno iza naredbe BL umetnu se podaci označeni labelama

• POZIVATELJ koristi labele da upiše argumente
– jednako kao pristup lib. lokacijama, ali je njihov položaj drugačiji
↳ nalaze se unutar naredaba i to „bliže" naredbi BL

```
; POZIVATELJ
...
STR   R5, PAR ; pripremi arg.
BL    FUNKC

PAR DW  0

STR   R0, REZ
....

FUNKC ; u LR je povratna adresa
; dohvat parametara
; pomak LR za 4 mjesta
; tijelo
MOV   PC, LR
```

funkcija čita parametre tako da za njihovo adresiranje koristi povratnu adresu zapisanu u LR

upravo LR pokazuje na param.

Primjer

```
MAIN  MOV   SP, #0x10000
      MOV   R1, #0x80000000
      MOV   R0, #0x35
      STR   R1, BZPH      ; spremi viši dio param.
      STR   R0, BZPL      ; spremi niži dio param
      BL    BZP2K

BZPL  DW 0
BZPH  DW 0
REZL  DW 0
REZH  DW 0

; mjesto nastavka izvođenja
      LDR   R0, REZL      ; kopiraj rezultat u memoriju
      LDR   R1, REZH
      MOV   R2, #REZ
      STMIA R2, {R0-R1}
      SWI   0x123456

REZ DW 0,0                ; mjesto za pohranu rezultata
```

BL → pc skok u funkciju, LR postaje ↓

↳ LR pokazuje ovdje → SPREMI „adresu iza BL-a" u LR

LR = adresa BZPL

↳ gdje ćemo vratiti rezultat

```
BZP2K STMFD SP! {R0-R1}
      LDR   R0, [LR], #4
      LDR   R1, [LR], #4
```

→ uzmi podatke sa memorijske adrese gdje LR pokazuje

↳ uvećaj LR nakon svakog čitanja

REGISTAR:	VRIJEDNOST	Što sadrži
LR	adresa BZPL	1. param = niži dio
LR + 4	adresa BZPH	2. param = viši dio